

# DEEP LEARNING, TINYML AND OPEN SOURCE

---

JONATHAN REYMOND

**EPFL**

# PLAN

- Lensless imaging
- Reconstruction methods
- Goals and motivations
- Model deployment
- Results
- Summary
- Next

# LENSLESS IMAGING : CONTEXT

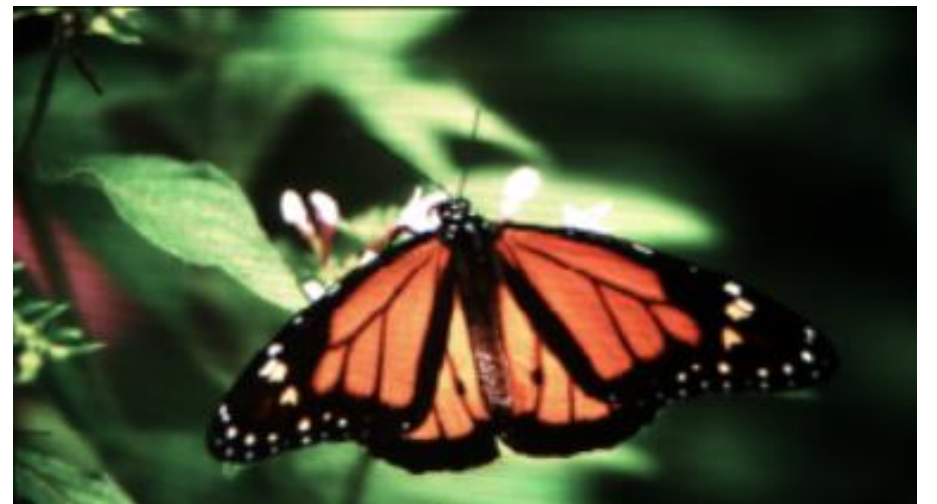


- Pros
  - High-resolution
  - Focalization
- Cons
  - Expensive
  - Weight
  - Space

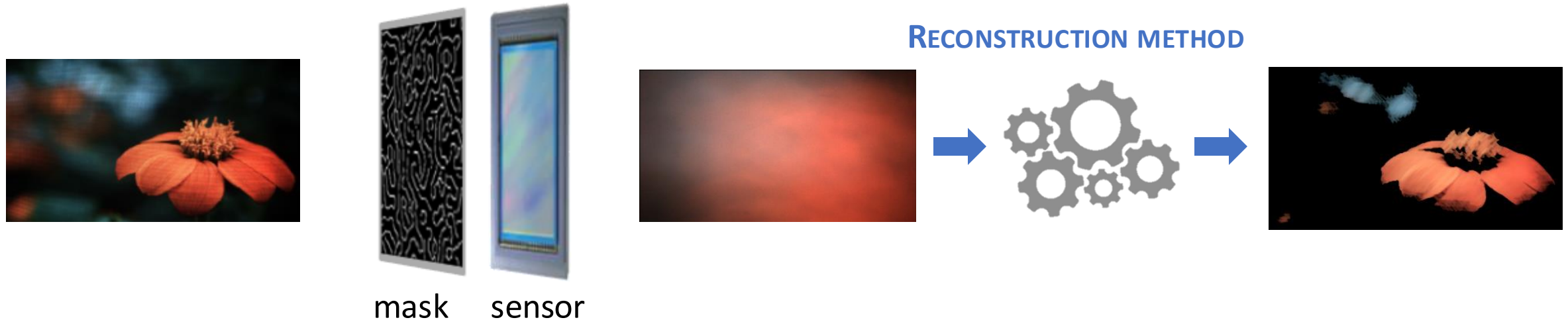
*What if remove lens ?*



RECONSTRUCT



# LENSLESS IMAGING: IDEA



## BENEFITS

- ✓ Low-cost, light-weight, space
- ✓ Miniaturization

## APPLICATIONS

- Medical systems
- Wearables
- IoT devices

# RECONSTRUCTION METHODS

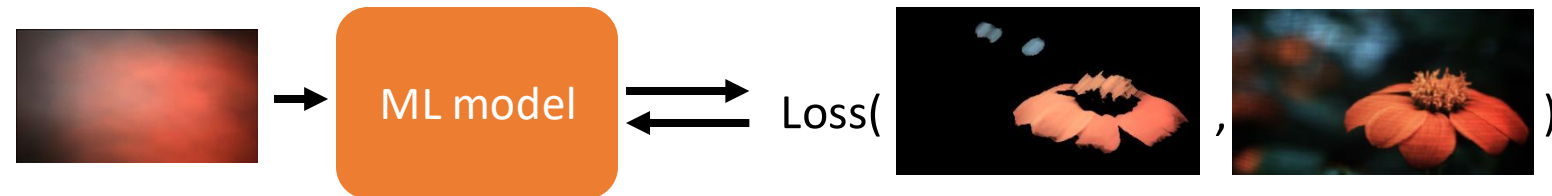
## 1. CLASSICAL ALGORITHMS

- Based signal processing theory
- Iterative algorithms
- FISTA, ADMM, gradient descent



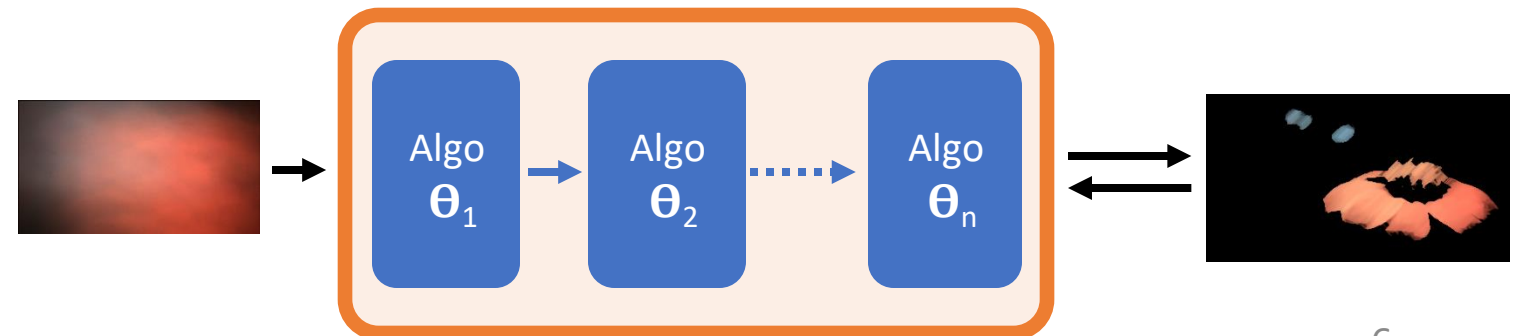
## 2. DEEP LEARNING

- Need dataset



## 3. UNROLLED OPTIMIZATION

- Mix classic + deep learning



# RECONSTRUCTION: PROJECT TARGET

- Study two deep learning approaches
  - Monakhova et al. (Learned reconstruction) + Khan et al. (FlatNet)
- Benefits of ML



Ground truth



ADMM



Unrolled ADMM



ML

*Running time on GPU (ms)*

1'520

71

10

# MONAKHOVA ET AL.

- Based on U-Net

- Loss

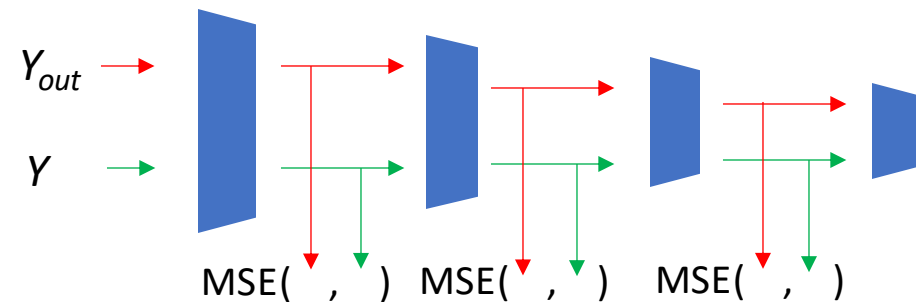
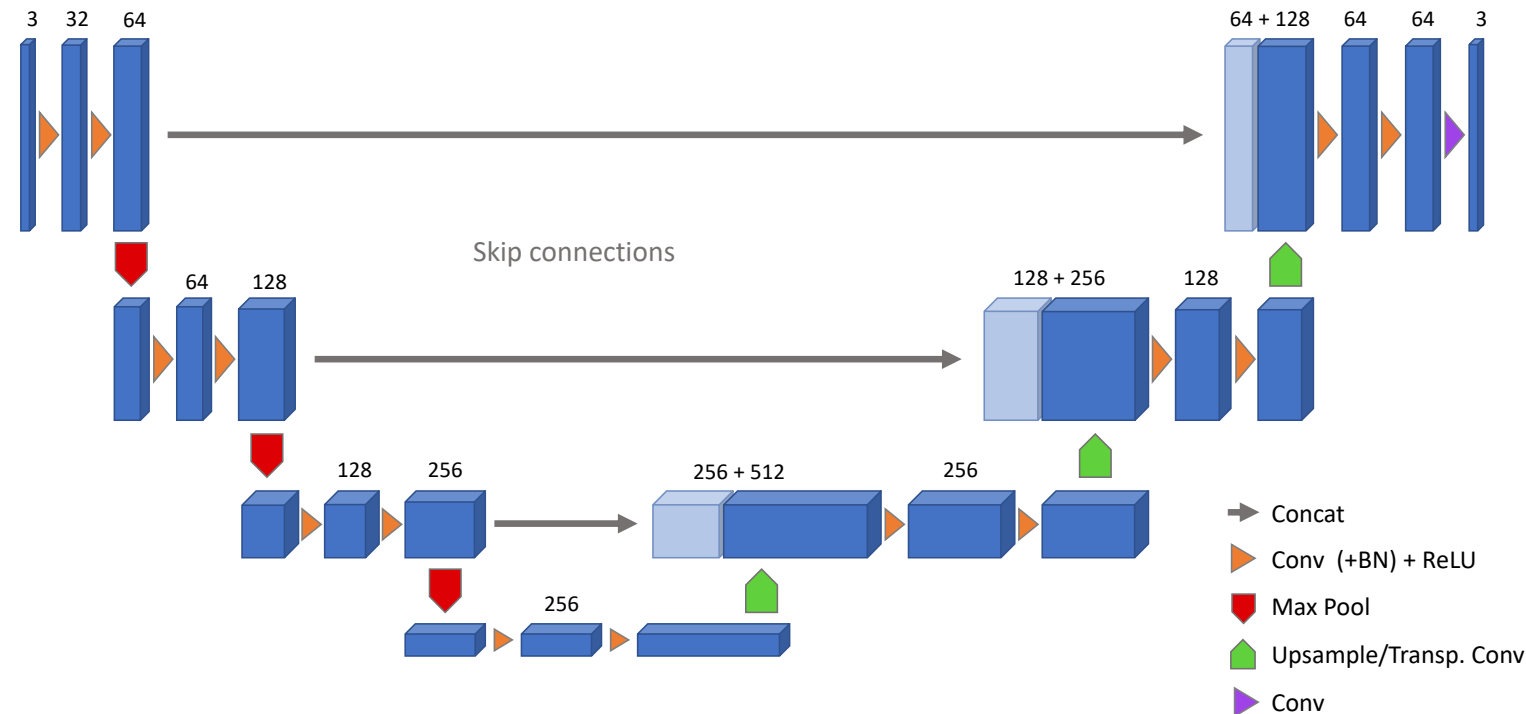
- Mean-squared error (MSE)

- $Loss(Y_{out}, Y) = \|Y_{out} - Y\|_2^2$

- Pixelwise comparison

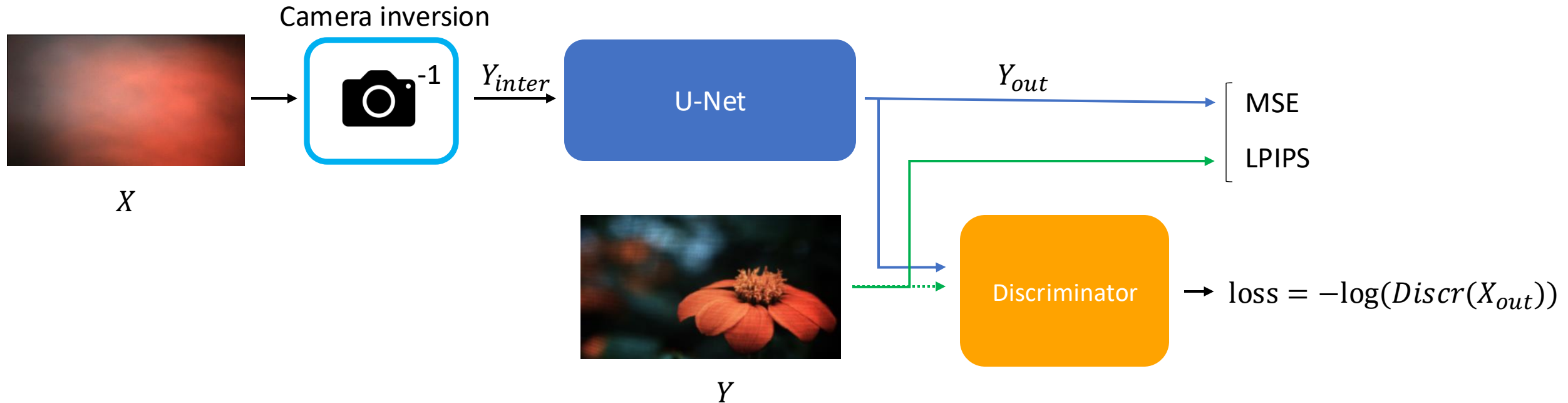
- LPIPS error

- Use pretrained model
    - Different models : AlexNet, VGG
    - Capture perception : edges, shapes





# KHAN ET AL.: OVERVIEW



# KHAN ET AL.: SETTING-UP

- Inverse problem formulation



$$X = H \cdot Y$$

- Invert  $H$  effect  $\rightarrow H^{-1}$
- $H^{-1}$  undetermined  
 $\rightarrow$  need simplification : **with mask**

# KHAN ET AL.: SETTING-UP

## SEPARABLE MASK

- Inner-product of 2 vectors

$$X = \underbrace{\Phi_L}_{} Y \underbrace{\Phi_R^T}_{}$$

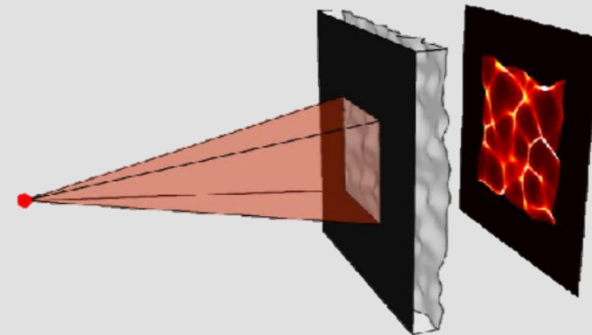
*Linear operators (matrix form)*

## NON-SEPARABLE MASK

- Assume PSF shift-invariant

$$X = Y * \underbrace{h}_{}$$

*PSF of camera*



# KHAN ET AL.: CAMERA INVERSION



*Idea : invert operation  $\rightarrow$  learnable layer*

## SEPARABLE MASK

$$X = \Phi_L Y \Phi_R^T$$

$$Y_{inter} = f(W_1 X W_2)$$

$\Phi_L$  adjoint  
 $\Phi_R^T$  adjoint

Non-linearity: leaky ReLU

## NON- SEPARABLE MASK

$$X = Y * h$$

$$\mathcal{F}(X) = \mathcal{F}(Y) \odot H$$

Pointwise multiplication

$$Y_{inter} = \mathcal{F}^{-1}(\mathcal{F}(W) \odot \mathcal{F}(X))$$

# MASTER THESIS: SUBJECT



## GOAL

- Implement Monakhova et al. + Khan et al. models
- Deploy to embedded systems



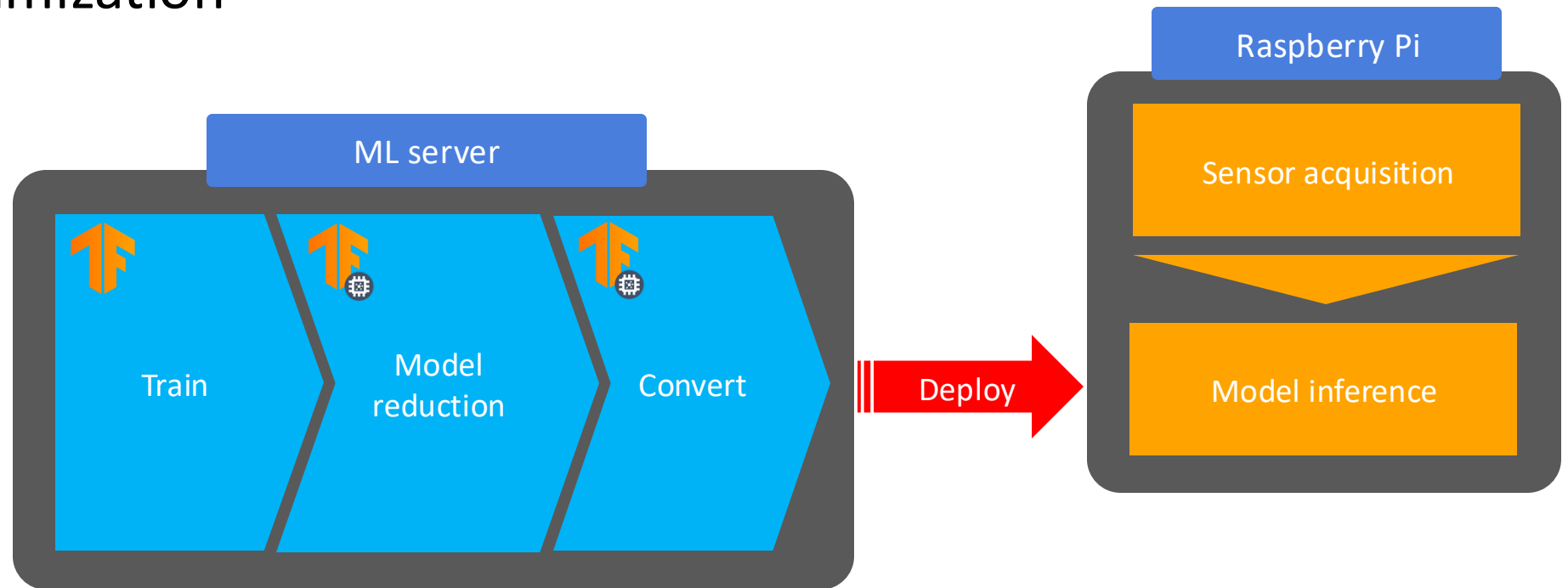
## MOTIVATIONS

- ↗ Reproducibility + availability
- ↗ Usable in various devices : phones, Raspberry Pi

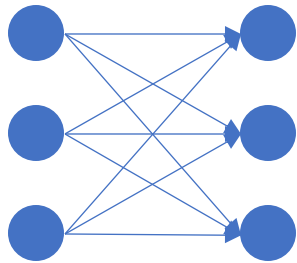
# WORKFLOW

- Need reduce size model for embedded systems
  - Flash : model parameters + architecture
  - RAM : computation, store temporary results

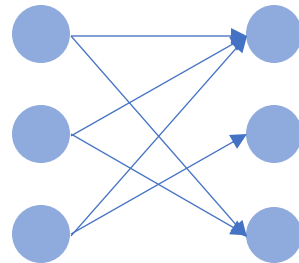
→ Model optimization



# MODEL OPTIMIZATION: WEIGHT PRUNING



|      |       |       |
|------|-------|-------|
| 3.32 | 29.01 | 27.07 |
| 0.9  | 50.3  | -21   |
| 8.9  | -0.1  | 7.9   |



|      |      |       |
|------|------|-------|
| 3.32 | 0    | 27.07 |
| 0.9  | 0    | -21   |
| 8.9  | -0.1 | 0     |

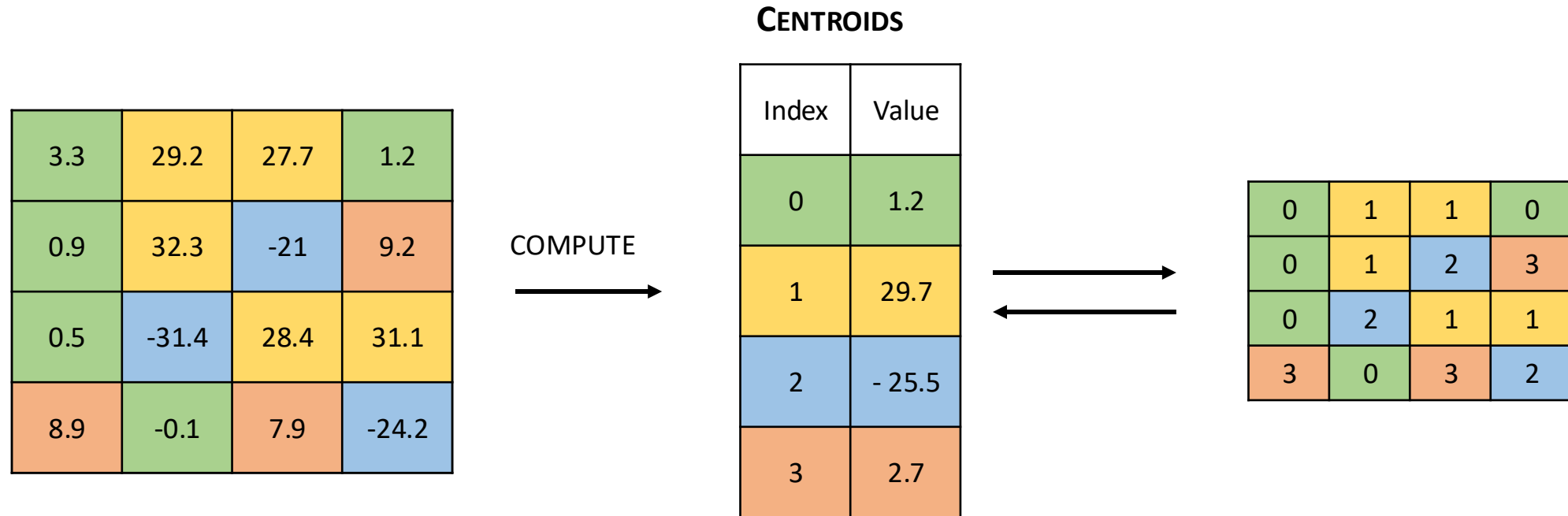
- Magnitude pruning

$$w_i = \begin{cases} w_i & \text{if } |w_i| > \lambda \\ 0 & \text{otherwise} \end{cases}$$

- Sensitivity pruning

$\lambda$  dependent to layer's std. deviation

# MODEL OPTIMIZATION: CLUSTERING



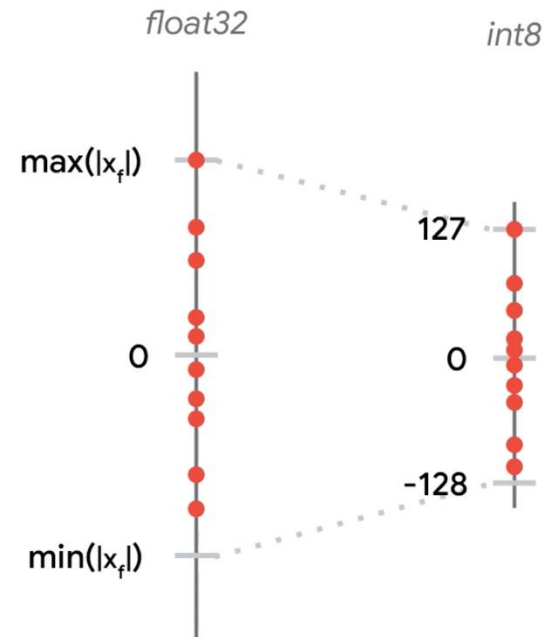


# MODEL OPTIMIZATION: QUANTIZATION



*Idea : floating point : 32 bits (float32)  $\rightarrow$  integer : 8 bits (int8)*

- Weight quantization:
  - Compress weights int8
  - Decompress + compute in float32
  - ~4x reduction Flash
- Inference quantization
  - Store + compute in int8
  - Quantization-aware training (QAT)
  - Reduction Flash + RAM

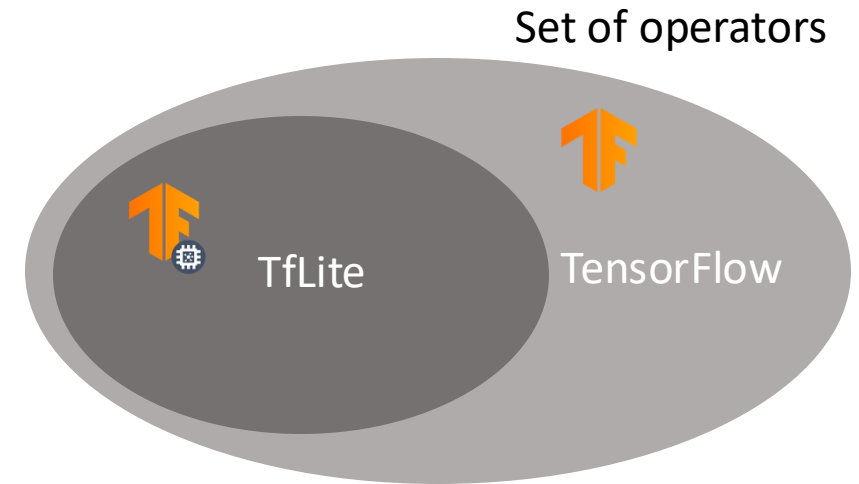


# MODEL OPTIMIZATION: TENSORFLOW LITE

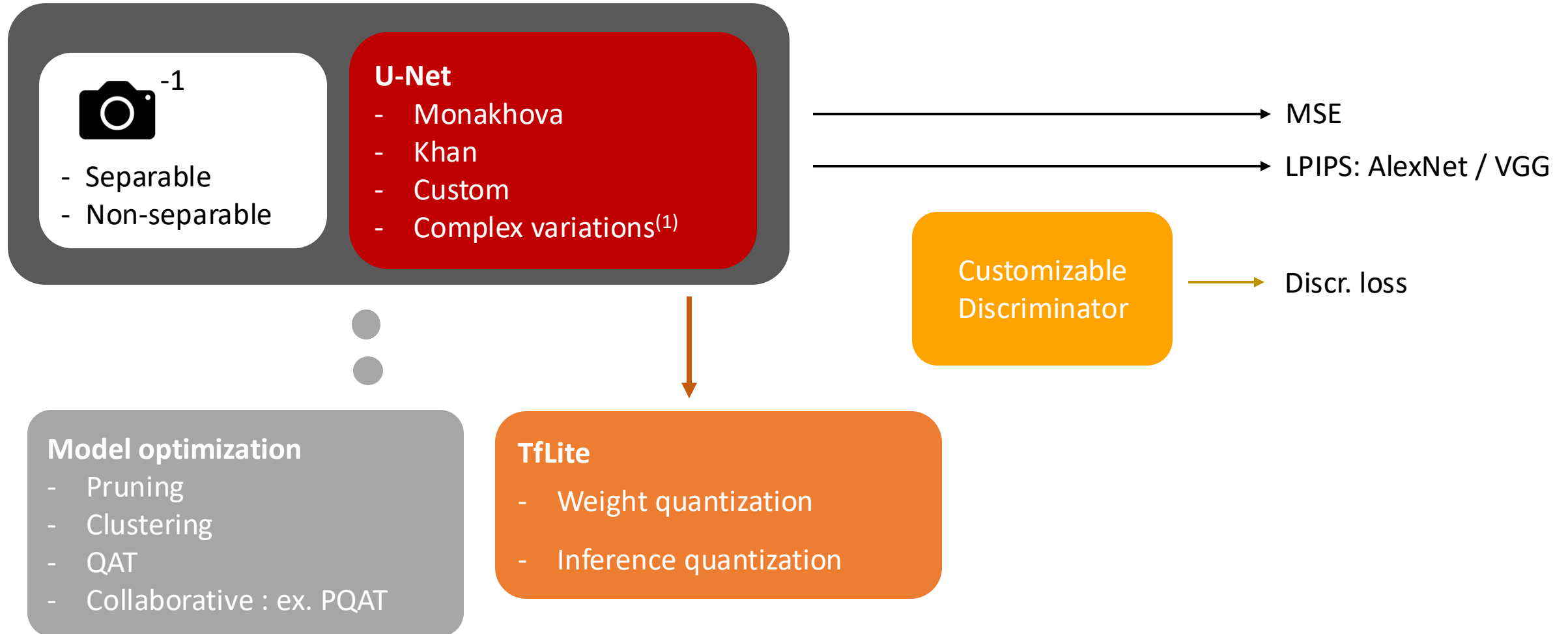
TensorFlow not for embedded systems

TfLite format : for Python, C++ apps

- Not hardware dependent
- No external tools needed



# IMPLEMENTATION: OVERVIEW



(1) "Keras-unet-collection" [2021, Y. Sha]

# RESULTS: MONAKHOVA ET AL.

- DiffuserCam dataset
  - 25'000 RGB images
  - Shift-invariant PSF (non-separable)



Raw measurements

Expected

Ours

Ground truth

LPIPS (lower ✓)

0.246

**0.231**

MSE (lower ✓)

**0.015**

0.038

# RESULTS: KHAN ET AL.

Ground truth



Ours



- FlatCam dataset
  - 10'000 images
  - Bayer measurements
  - Separable mask

|                 |  | Expected    | Ours         |
|-----------------|--|-------------|--------------|
| PSNR (higher ✓) |  | 19.62       | <b>19.80</b> |
| SSIM (higher ✓) |  | <b>0.64</b> | 0.53         |



# RESULTS: FLATNET

DiffuserCam (non-separable) + inversion layer

Expected



Ours



Ground truth



# RESULTS: MODEL REDUCTION

Ground truth



Normal



Weight quantization



Pruned



Inference quantization



# RESULTS: MODEL REDUCTION

|                  | Normal       | Pruned <u>+ quant</u> | QAT         |
|------------------|--------------|-----------------------|-------------|
| LPIPS            | <b>0.38</b>  | 0.41                  | 0.46        |
| MSE              | <b>0.039</b> | 0.045                 | 0.095       |
| Flash (zip, MB)  | 40.1         | 25 <b>→ 7</b>         | <b>9.4</b>  |
| RAM (MB)         | 320          | 320                   | <b>130</b>  |
| Time (1 core, s) | 1.74         | 1.56                  | <b>1.39</b> |



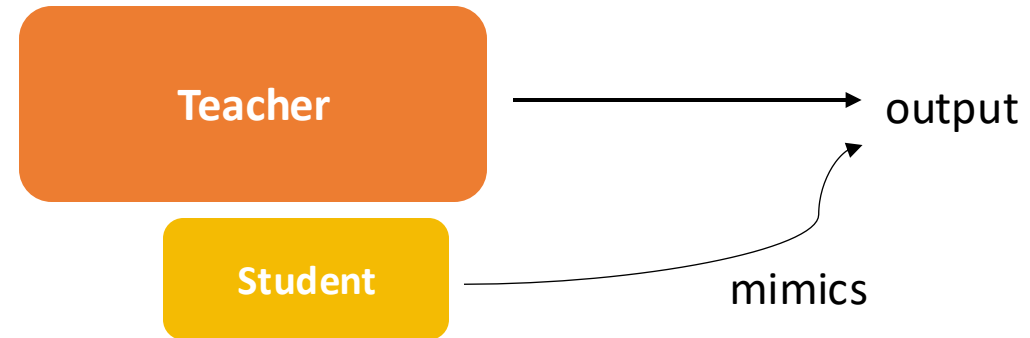
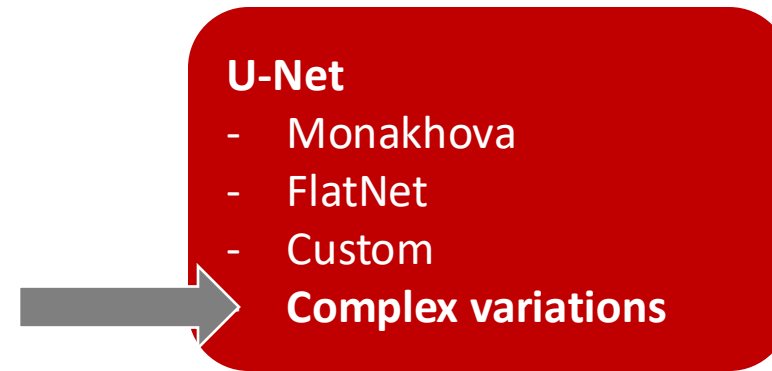


# SUMMARY

- Study Monakhova et al. + Khan et al.
- Deployment
  - Model optimization
  - TensorFlow Lite
- Importance of inversion layer
- Effects of model optimization

# NEXT

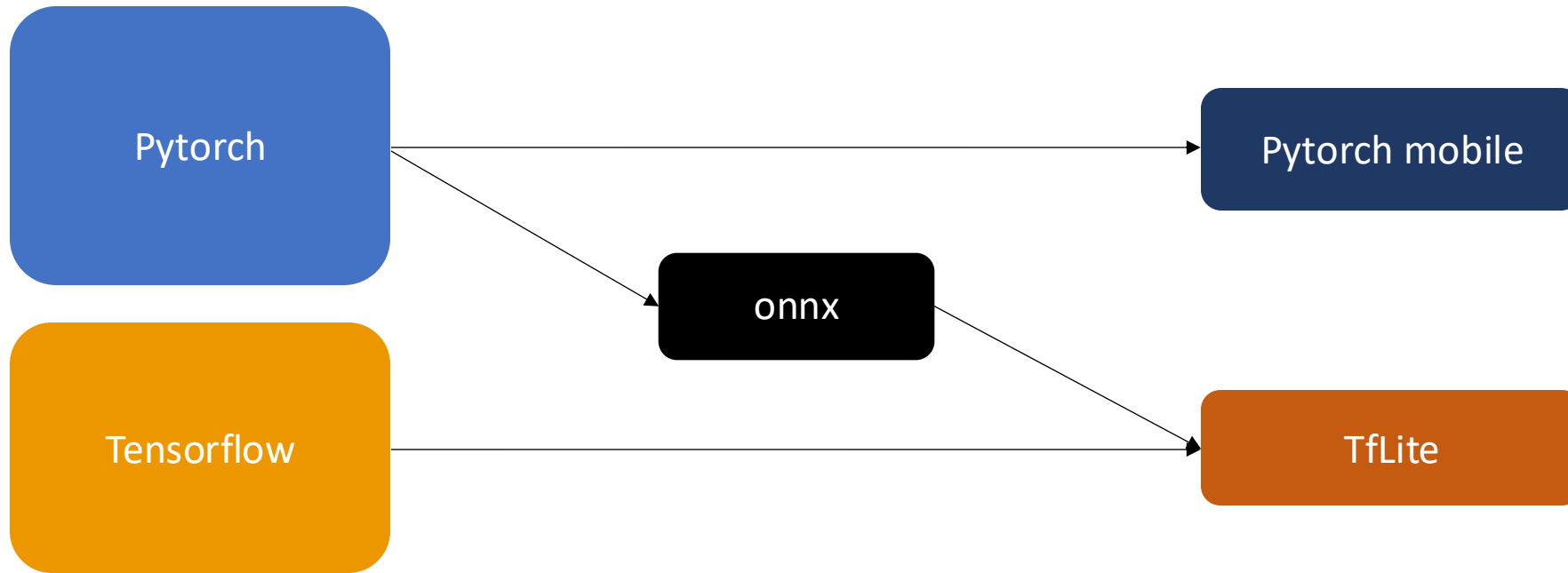
- Test more complex models
- Knowledge distillation
- Deploy image classification models





THANK YOU  
*for your attention*

# WHY TENSORFLOW?



- Less model optimization techniques (ex. clustering)
- Unknown operators during conversion

# MODEL OPTIMIZATION: QUANTIZATION

- Idea : floating point : 32 bits (float32) → integer : 8 bits (int8)
- Weight quantization:

- Compress weights int8:

$$x_{int} = \frac{x_{float} - min_f}{max_f - min_f} * 255 - 128$$

- Decompress + compute in float32:
- ~4x reduction Flash

$$x_{float} = \frac{x_{int} - min_f}{255} * (max_f - min_f) + min_f$$

- Inference quantization

- Store + compute in int8
- Can be done in training : Quantization-aware training (QAT)
- ~4x reduction Flash + RAM

